JDBC API

What is JDBC API?

JDBC ဆိုတာက Java programming မှာ Database တွေနဲ့ ချိတ်ဆက်ပြီး အလုပ်လုပ်နိုင်ဖို့ ထောက်ပံ့ပေးထားတဲ့ **Application Programming Interface (API)** တစ်ခုပါ။ ဒီ API ကို Sun Microsystems (အခု Oracle လက်အောက်မှာ ရှိတယ်) က ဖန်တီးခဲ့ပြီး Java Standard Edition (Java SE) ရဲ့ တစ်စိတ်တစ်ပိုင်းအနေနဲ့ ပါဝင်ပါတယ်။ JDBC က Developersတွေကို Relational Database တွေနဲ့ အလွယ်တကူ ချိတ်ဆက်ပြီး ဒေတာတွေကို ထိန်းချုပ်နိုင်တဲ့ API တစ်မျိုးပါ။ ဥပမာ - MySQL, PostgreSQL, Oracle Database, Microsoft SQL Server စတဲ့ Database တွေနဲ့ ချိတ်ဆက်လို့ ရပါတယ်။

JDBC ရဲ့အဓိကအစိတ်အပိုင်းများ

JDBC ရဲ့အလုပ်လုပ်ပုံကို နားလည်ဖို့ အရင် အဓိကအစိတ်အပိုင်းတွေကို အရင်သိထားရပါမယ်။

Driver Manager

- ဒေတာဘေ့စ်နဲ့ ချိတ်ဆက်ဖို့ Connection တစ်ခုကို ဖန်တီးပေးတဲ့ Class တစ်ခုပါ။
- သူက JDBC Driver တွေကို စီမံခန့်ခွဲပြီး သက်ဆိုင်ရာ Database နဲ့ ချိတ်ဆက်ပေးပါတယ်။

Connection

- ဒေတာဘေ့စ်တစ်ခုနဲ့ ချိတ်ဆက်မှုကို ကိုယ်စားပြုတဲ့ Interface တစ်ခုပါ။
- Connection မရှိရင် ဒေတာဘေ့စ်နဲ့ ဘာမှ လုပ်လို့မရပါဘူး။

Statement

- SQL မေးခွန်းတွေ (Queries) ကို ဒေတာဘေ့စ်ဆီ ပို့ပေးဖို့ အသုံးပြုတဲ့ Interface တစ်ခု။
- ဥပမာ - SELECT, INSERT, UPDATE, DELETE စတဲ့ SQL ကုဒ်တွေကို လုပ်ဆောင်နိုင်တယ်။

PreparedStatement

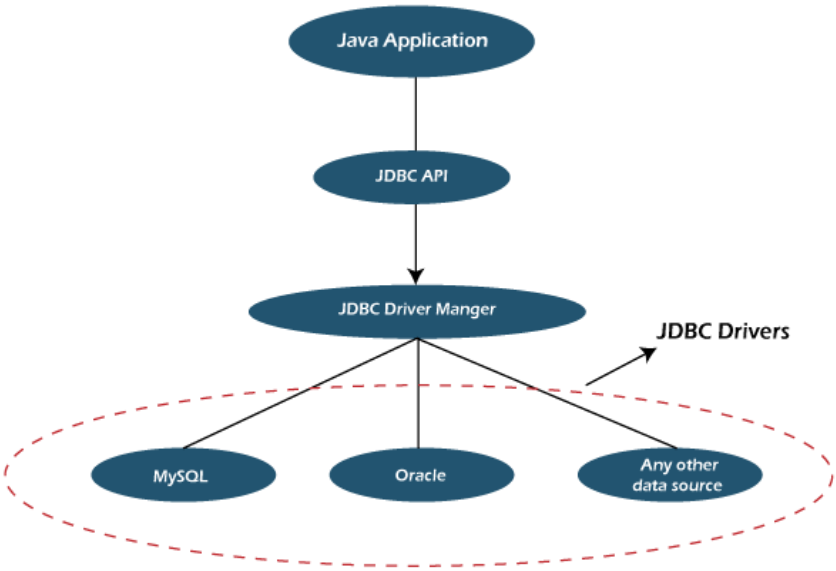
- Statement ရဲ့နောက်ထပ် ဗားရှင်းတစ်ခုလို့ ပြောရမယ်။
- ကြိုတင်ပြင်ဆင်ထားတဲ့ dynamic Query တွေကို ထည့်ပြီး Parameter တွေနဲ့ အလုပ်လုပ်နိုင်တယ်။ ပိုမြန်ပြီး လုံခြုံတယ်။
- (SQL Injection ကို ကာကွယ်ပေးတယ်)။

ResultSet

- SQL Query တစ်ခုကို လုပ်ဆောင်ပြီးရင် database ကနေ ပြန်လာတဲ့ ရလဒ်တွေကို သိမ်းထားတဲ့ Object တစ်ခုပါ။
- ဥပမာ - SELECT Query တစ်ခုရဲ့ရလဒ်ကို Row နဲ့ Column အနေနဲ့ ဖတ်ယူနိုင်တယ်။

JDBC Driver

- Database တစ်ခုချင်းစီအတွက် သီးသန့် Driver လိုအပ်ပါတယ်။
- ဥပမာ - MySQL အတွက် mysql-connector-java ၊ PostgreSQL အတွက် postgresql-jdbc စသဖြင့် Driver တွေ ရှိပါတယ်။



1. JDBC Driver ကို ထည့်သွင်းပြီး Register လုပ်ခြင်း

- **ဘာလုပ်တာလဲ?** သက်ဆိုင်ရာ Database ရဲ့ JDBC Driver ကို ကျတော်တို့ရဲ့ Java ပရိုဂရမ်ထဲ ထည့်သွင်းပြီး Driver Manager မှာ မှတ်ပုံတင်ရပါတယ်။
- **ဘယ်လိုလုပ်လဲ?** အရင်က ***Class.forName("com.mysql.jdbc.Driver");*** လို့ ရေးပြီး မှတ်ပုံတင်ခဲ့ရပေမယ့် Java 7 နဲ့ အထက်မှာတော့ Driver ကို အလိုအလျောက် Register လုပ်ပေးပါတယ်။ Driver JAR ဖိုင်ကို Project ထဲထည့်ရုံနဲ့ လုံလောက်ပါတယ်။
- **ဥပမာ**: MySQL အတွက် mysql-connector-java-x.x.xx.jar ကို Project ထဲ ထည့်ရပါတယ်။

2. Database နဲ့ Connection ချိတ်ဆက်ခြင်း

- **ဘာလုပ်တာလဲ?** Database နဲ့ ချိတ်ဆက်ဖို့ Connection Object တစ်ခု ဖန်တီးရပါတယ်။
- **ဘယ်လိုလုပ်လဲ?** **DriverManager.getConnection() Method** ကို သုံးပြီး **Database URL, Username** နဲ့ **Password** ထည့်ပေးရပါတယ်။

Sample code :

```
String url          = "jdbc:mysql://localhost:3306/jdbc_database";
String username     = "root";
String password     = "password";
Connection connection = DriverManager.getConnection(url, username, password);
```

၃. SQL Statement ဖန်တီးခြင်း

- **ဘာလုပ်တာလဲ?** Database ကို မေးခွန်းထုတ်ဖို့ Statement ဒါမှမဟုတ် PreparedStatement Object တစ်ခု ဖန်တီးရပါတယ်။
- **ဘယ်လိုလုပ်လဲ?** Connection Object ကနေ createStatement() ဒါမှမဟုတ် prepareStatement() Method ကို သုံးပါတယ်။

Sample code

```
Statement statement = connection.createStatement();
```

ဒါမှမဟုတ် Dynamic Query တွေအတွက်ဆိုရင်တော့,

```
String sql = "SELECT * FROM users WHERE id = ?";
PreparedStatement pstmt = connection.prepareStatement(sql);
pstmt.setInt(1, 10);
```

4. SQL Query ကို Execute လုပ်ခြင်း

- **ဘာလုပ်တာလဲ?** SQL မေးခွန်းကို Database ဆီ ပို့ပြီး လုပ်ဆောင်ခိုင်းပါတယ်။
- **ဘယ်လိုလုပ်လဲ?**
  - Data ဖတ်ဖို့ဆိုရင်: executeQuery() ကို သုံးပြီး ResultSet ရယူပါတယ်။
  - Data ထည့်တာ၊ ပြင်တာ၊ ဖျက်တာအတွက်: executeUpdate() ကို သုံးပါတယ်။

Sample code

```
ResultSet resultSet = statement.executeQuery("SELECT * FROM users");    // read

int insert = statement.executeUpdate("INSERT INTO users (name, age) VALUES ('Aung', 25)");  // insert
```

5. ရလဒ်ကို ဖတ်ရှုခြင်း (ResultSet ကိုအသုံးပြုတာ)

- **ဘာလုပ်တာလဲ?** Database ကနေ ပြန်လာတဲ့ ဒေတာတွေကို ResultSet ကနေ ဖတ်ရပါတယ်။
- **ဘယ်လိုလုပ်လဲ?** next() Method နဲ့ Row တစ်ခုချင်း ရွှေ့ပြီး getxxx() Method တွေ (ဥပမာ getString(), getInt()) နဲ့ ဒေတာတွေ ထုတ်ယူပါတယ်။

Sample code

```
while (resultSet.next()) {

   String name = resultSet.getString("name");

   int age = resultSet.getInt("age");

   System.out.println("Name: " + name + ", Age: " + age);

}
```

၆. Resources တွေကို ပိတ်ခြင်း

- **ဘာလုပ်တာလဲ?** သုံးပြီးသွားတဲ့ ResultSet, Statement, Connection စတဲ့ Object တွေကို ပိတ်ပေးရပါတယ်။ မပိတ်ရင် Memory Leak ဖြစ်နိုင်ပါတယ်။
- **ဘယ်လိုလုပ်လဲ?** close() Method ကို သုံးပါတယ်။ Java 7 နဲ့အထက်မှာ try-with-resources နဲ့ အလိုအလျောက် ပိတ်ပေးလို့ရပါတယ်။

JDBC ရဲ့အလုပ်လုပ်ပုံ အကျဉ်းချုပ်

1. Driver ထည့်ပြီး Register လုပ်။
2. Database နဲ့ Connection ချိတ်။
3. SQL Query ဖန်တီးပြီး Statement ပြင်ဆင်။
4. Query ကို Database မှာ Execute လုပ်။
5. လိုအပ်ရင် ResultSet ကနေ ဒေတာဖတ်။
6. Connection နဲ့ Resources တွေ ပိတ်။